Quiz #2

Answer the following questions. Use your own words while answering the questions. Write your answers in a Word document and upload your solutions (.doc or .pdf file) to the ALMS system until May 17, 2020 23:59.

1. Write the types of each variables in the below code segment according to their lifetimes and explain when memory is allocated and deallocated for each variable. (Hint: Assume that the programming language used for the below code segment is a new language which can support variable types in both C and JavaScript.)

```
int a;
static float b;
int main( ) {
   int A[100], *p;
   static double x;
   …
   p = (int *)malloc(sizeof(int)*1000);
   …
   i = “This is a string”;
   …
   i = A[5];
   …
   free(p);
}
```

2. Consider the following skeletal C program:

```
void fun1(void);   // function prototype
void fun2(void);   // function prototype
void fun3(void);   // function prototype

void main(void){
   int a, b, c;
   //  1
   fun1( );
}

void fun1(void){
   int x, a, b;
   //  2
   fun2( );
}

void fun2(void){
   int y, b, c;
   //  3
   fun3( );
}
```

```
void fun3(void){
   int z, c, i;
   …
   //  4
}
```

Write the referencing environments for the lines 1, 2, 3, and 4; if
a) static scope is used
b) dynamic scope is used.

3. Assume that you are writing a C program that copies a string variable `str1` to another string variable `str2` by using `strcpy` function (you call `strcpy(str2,str1)`). You should define `str1` and `str2` as character arrays, and assume that the length of `str1` is 10 characters, and length of `str2` is 5 characters. Print the value of `str2` after the copy operation. Write a simple C code that performs the above copy operation. Run your code, print the output. Explain the problems in the above operations (if any). Then, re-write the same code with C++ by using the `string` class. Compare and discuss the similarities and differences between the two implementations.

4. Assume the following rules of associativity and precedence for expressions:

Precedence	Highest	++, --
		*, /, not
		+, – , &, mod
		Unary –
		== , != , < , <= , > , >=
		and
	Lowest	or, xor
Associativity	Left-to-right	

Show the order of evaluation of the following expressions by parenthesizing all subexpressions and placing a superscript or subscript on the right parenthesis to indicate the order.

++a – b / c + d--

a * b == -c + d mod 5 or x != y